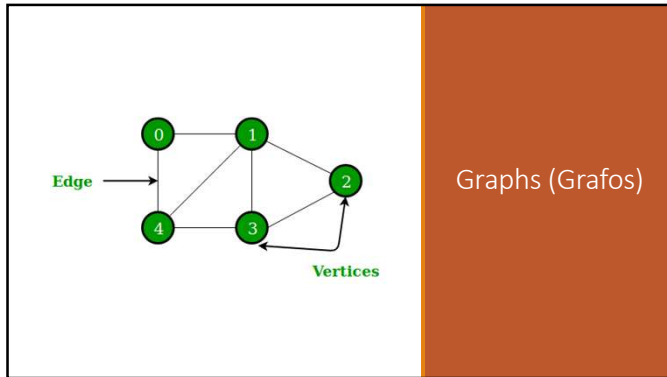




1



2

Grafos

- Los grafos sirven para modelar relaciones
 - Lo hacen mejor que casi cualquier otra estructura de datos.

3

Grafos

Un **grafo** en estructuras de datos es una colección de **nodos** (o **vértices**) conectados por **aristas**.

Los grafos se usan mucho para representar relaciones entre objetos, como:

- **mapas**
- **Redes sociales:** Representan conexiones entre personas.
- **Sistemas de transporte:** Modelan rutas entre lugares (por ejemplo, mapas).
- **Rastreadores web:** Representan sitios web como un grafo de enlaces.
- **Redes de computadoras:** Modelan conexiones entre computadoras.
- **Problemas de planificación:** Representan tareas y dependencias.

4

Componentes de un grafo

Vértices (Nodos):

- Son las unidades principales del grafo.
- Representan las entidades dentro de la relación.
- Ejemplo: En una red social, los vértices pueden representar personas.

5

5

Componentes de un grafo

Aristas:

- Representan las conexiones o relaciones entre los vértices.
- Pueden ser:
 - **Dirigidas:** Conexión de un solo sentido (por ejemplo, una calle de una sola vía).
 - **No dirigidas:** Conexión de doble dirección (por ejemplo, una amistad mutua en redes sociales).

6

6

Grafos en Java

Java no tiene una clase de librería estándar incorporada específicamente para grafos (como lo hace para listas o mapas), pero puede implementar grafos utilizando colecciones Java existentes o usar librerías externas que proporcionen estructuras de datos de grafos y algoritmos

7

7

Grafos en Java

1. Hacerlo usted mismo con Colecciones Java (Implementación Manual)

Puede implementar grafos usando:

- **Lista de adyacencia:** Usar `Map<Node, List<Node>>`
- **Matrix de adyacencia:** Usar 2D array , por ejemplo `int[][]`

8

8

Grafos en Java

2. Usar una librería

Si busca una funcionalidad de grafos sólida:

- JGraphT : <https://jgrapht.org/>

9

El Grafo ADT

- Un grafo es una estructura no lineal que consta de dos sets:
 - V : Set de vértices. Estos son los nodos que contienen los datos.
 - E : Set de las aristas. Muestran cómo se relacionan los vértices. Representar rutas/conexión entre los nodos.
- Ambos sets deben ser finitos.
- Cualquiera de los dos set puede estar vacío, pero si set V está vacío entonces set E también debe estar vacío.
- *Lo que significa la relación entre nodos depende de lo que estemos representando.*

10

Terminología de grafos

11

Terminología de grafos

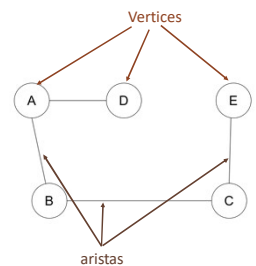
Visualmente:

- Los vértices (nodos) se representan como círculos o puntos etiquetados.
- Las aristas se representan como segmentos de línea que conectan los vértices.

También podemos representar el grafo como:

$$V = \{A, B, D, C, E\}$$

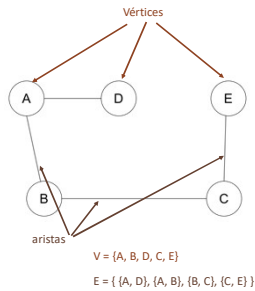
$$E = \{ \{A, D\}, \{A, B\}, \{B, C\}, \{C, E\} \}$$



12

Terminología de grafos

- **Vertices** es un conjunto de los valores del grafo.
- **arista** es un conjunto de los pares de los nodos que están conectados.
- Si hay una arista entre los nodos A y B, entonces hay un camino de A a B, y viceversa.
- Se dice que los nodos conectados por una arista son **adyacentes**.



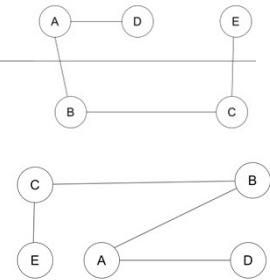
13

El diseño visual del grafo es irrelevante:

$V = \{A, B, D, C, E\}$

$E = \{ \{A, D\}, \{A, B\}, \{B, C\}, \{C, E\} \}$

¡Ambas figuras representan el mismo grafo!

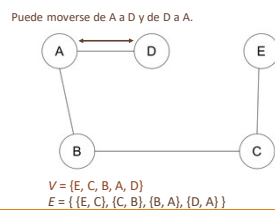


14

Grafos no dirigidos vs dirigidos

No dirigidos:

- Este es un **grafo no dirigido**, o simplemente un **grafo**.
- Las aristas no tienen dirección. Podemos movernos de cualquier manera a través de los nodos.
- Teniendo en cuenta esto, las aristas son pares desordenados. Por lo tanto, $\{A, D\}$ y $\{D, A\}$ representan la misma arista.
- **Ejemplo:** Una red social donde la amistad es mutua

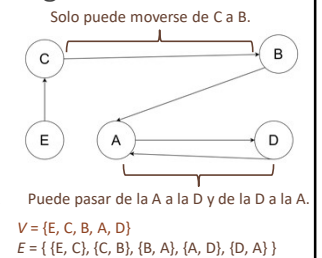


15

Grafos no dirigidos vs dirigidos

Dirigidos:

- Este es un **grafo dirigido** o un **dígrafo**.
- Las aristas se representan con flechas.
- Indican en la dirección en la que podemos movernos por el grafo.
- Las aristas en este caso son pares ordenados, ya que el orden es relevante.
- Ejemplos: Una red de carreteras con calles de una sola vía, enlaces páginas web

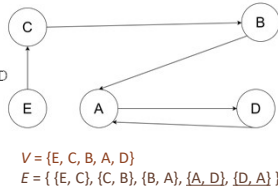


16

Grafos no dirigidos vs dirigidos

Dirigidos:

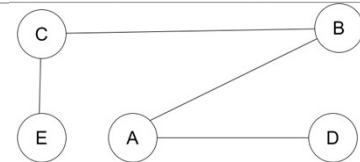
- Las aristas se pueden ver como: **{origen, destino}**
- Aquí $\{A, D\}$ y $\{D, A\}$ no son lo mismo.
 - Uno significa que podemos movernos de A a D.
 - La otra manera en que podemos movernos de D a A.
- Para indicar que podemos movernos en una dirección, tenemos que incluir:
 - Visualmente: Ambas flechas en el grafo.
 - Estructura: Ambas parejas en el set E.



17

Grafos no dirigidos vs dirigidos

No dirigidos



$$V = \{E, C, B, A, D\}$$

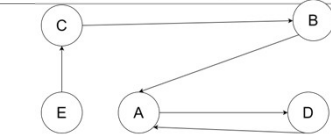
$$E = \{ \{E, C\}, \{C, B\}, \{B, A\}, \{D, A\} \}$$

Podemos movernos de un nodo a otro siempre y cuando estén conectados.

18

Grafos no dirigidos vs dirigidos

Dirigido



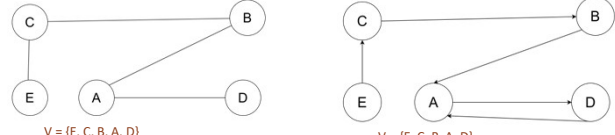
$$V = \{E, C, B, A, D\}$$

$$E = \{ \{E, C\}, \{C, B\}, \{B, A\}, \{A, D\} \}$$

Solo podemos movernos de un nodo a otro si la arista va en esa dirección.
En este grafo **no** podemos representar la arista entre los nodos E y C como $\{C, E\}$.

19

Grafos no dirigidos vs dirigidos



$$V = \{E, C, B, A, D\}$$

$$E = \{ \{E, C\}, \{C, B\}, \{B, A\}, \{D, A\} \}$$

$$V = \{E, C, B, A, D\}$$

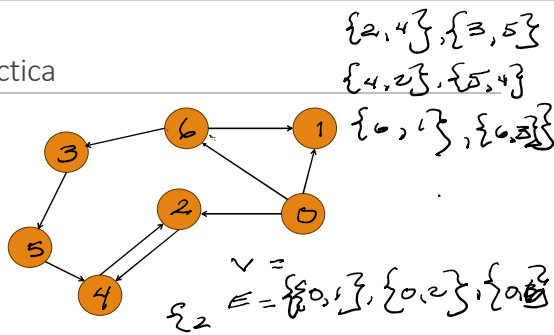
$$E = \{ \{E, C\}, \{C, B\}, \{B, A\}, \{A, D\}, \{D, A\} \}$$

A pesar de que estos grafos tienen estructuras similares, difieren mucho en:

- La forma en que podemos acceder a los nodos.
- ¿Cómo se debe presentar E?

20

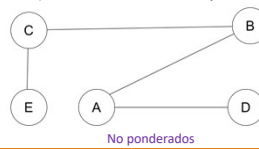
Práctica



21

Grafos ponderados frente a no ponderados
(Weighted Graph vs Unweighted Graph)

- Las aristas de un grafo pueden tener un peso asociado.
- A esto se le llama grafo ponderado.
- Lo que significa el peso depende de lo que estemos representando. (distancia, costo, tiempo, etc.)

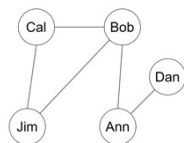


22

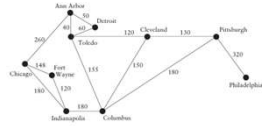
Tipos de grafos

El tipo de grafo que debemos usar depende mucho de lo que queramos modelar (representar).

Grafos no dirigidos:



V = Gente
E = Representa a los que son amigos



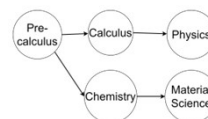
V = Ciudades
E = Qué ciudades están conectadas y su distancia

23

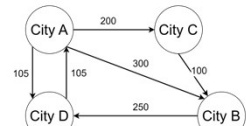
Tipos de grafos

El tipo de grafo que debemos usar depende mucho de lo que queramos modelar (representar).

Grafos dirigidos:



V = Curso
E = Las dependencias entre cursos



V = Ciudades
E = Sentido de las carreteras entre ciudades y sus distancias

24

Rutas y Ciclos

- Una de las razones por las que estudiamos grafos es para encontrar caminos entre vértices.

Las vías se pueden describir como::

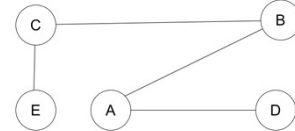
- Adyacente:** Cuando hay una arista que conduce a ese vértice.
- Camino:** Secuencia de vértices, donde cada nodo sucesivo es adyacente a su predecesor.
- Camino sencillo:** Ruta en la que los vértices son distintos, solo el primer y el último vértice pueden ser iguales. Si lo son, se llama ciclo.

25

25

Rutas y Ciclos

Adyacente: Cuando hay una arista que conduce a ese vértice.



Aquí C es adyacente a E y B.

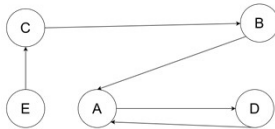
Del mismo modo, B es adyacente a C y A.

26

26

Rutas y Ciclos

Adyacente: Cuando hay una arista que conduce a ese vértice.



Dado que este grafo está dirigido, C es adyacente a E, pero C no es adyacente a B.

Del mismo modo, E no es adyacente a C.

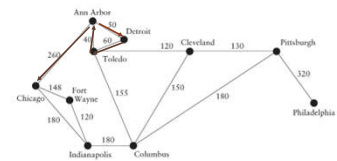
El vértice **x** es adyacente a **y**, si y puede llegar a x, **no al revés**.

27

27

Rutas y Ciclos

Ruta: Secuencia de vértices, donde cada nodo sucesivo es adyacente a su predecesor.



Una ruta:

Ann Arbor -> Detroit -> Toledo -> Ann Arbor -> Chicago

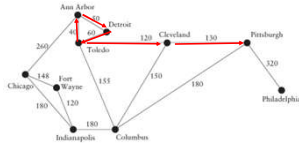
Tenga en cuenta que los vértices pueden repetirse.

28

28

Rutas y Ciclos

Ruta simple: Ruta donde los vértices son distintos, solo el primer y el último vértice pueden ser iguales. Si lo son, se llama ciclo.



Una ruta simple: Ann Arbor -> Detroit -> Toledo -> Cleveland -> Pittsburgh

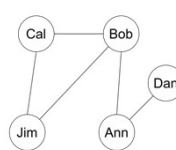
Un ciclo: Ann Arbor -> Detroit -> Toledo -> Ann Arbor

29

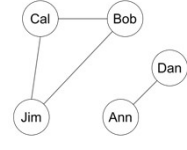
29

Grafos conectados

- Un grafo no dirigido está conectado si cada vértice tiene una ruta a todos los demás vértices del grafo.
- De lo contrario, se denomina Grafo inconexo.



Grafo conectado



Grafo inconexo

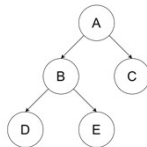
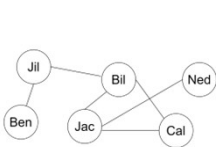
Puede llegar a todos los vértices desde cualquier nodo. Ann y Dan no pueden acceder a los vértices Jim, Cal y Bob.

30

30

Grafos y árboles

- Los grafos son la estructura de datos más general que estudiaremos.
- Nos permiten representar cualquier tipo de relación entre datos.
- Los árboles son un tipo especial de grafo.
 - Son grafos que están conectados y no tienen ciclos.

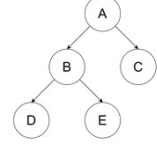
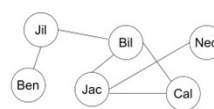


31

31

Grafos y árboles

- Nota:** Todos los árboles son grafos, pero no todos los grafos son árboles.



32

32

Grafos y árboles

Árboles	Grafos
Sin ciclos	Puede contener ciclos
Jerárquico	Conexiones generales
Una raíz	No se requiere raíz
Exactamente un camino entre nodos	Múltiples caminos posibles

33

Implementaciones de grafos

34

Representación de grafos

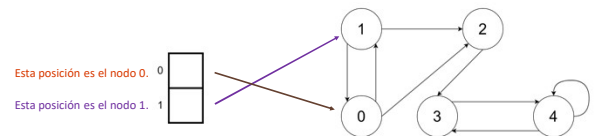
Las formas más comunes de implementar un grafo son:

- **Lista de adyacencia**— Usamos una matriz de listas. Cada lista almacena los vértices adyacentes a un vértice en particular.
 - **Ventajas**
 - Eficiente en memoria
 - Más comúnmente utilizado
- **Matriz de adyacencia**: utilizamos una matriz de dos dimensiones de tamaño $|V| \times |V|$.
 - **Ventajas**
 - Búsqueda rápida
 - **Desventajas**
 - Utiliza mucha memoria

35

Representación de grafos

- Para simplificar, usaremos grafos que van de 0 a N-1.
- Supongamos para ambas representaciones que:
 - La posición de cada vértice en la matriz es su valor respectivo.
 - El vértice 0 está en la posición 0, etc.

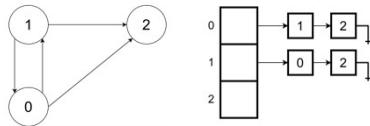


36

Listas de adyacencia

Listas de adyacencia

- Concepto similar al chaining separado en HashTables.
- Tenemos una matriz de tamaño N , donde N es el número de vértices en el grafo. ($N = |V|$)
- Cada posición es una lista, y cada posición representa un vértice del grafo.



37

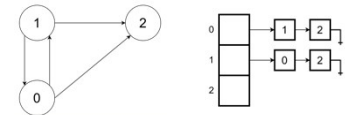
37

Listas de adyacencia

- Cada lista contiene los vértices a los que se puede llegar a través de ese vértice.
- Por lo tanto, cada lista contiene los vértices que son adyacentes a ella.

Ejemplo:

- El vértice 2 es adyacente a 1
- Por lo tanto, 2 está en la lista para el vértice 1.

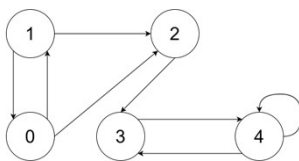


38

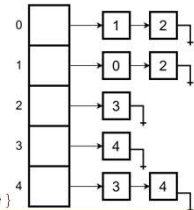
38

Listas de Adyacencia (Dirigidas)

Grafo:


 $V = \{0, 1, 2, 3, 4\}$
 $E = \{(0, 1), (0, 2), (1, 0), (1, 2), (2, 3), (3, 4), (4, 3), (4, 4)\}$

Lista de adyacencia:

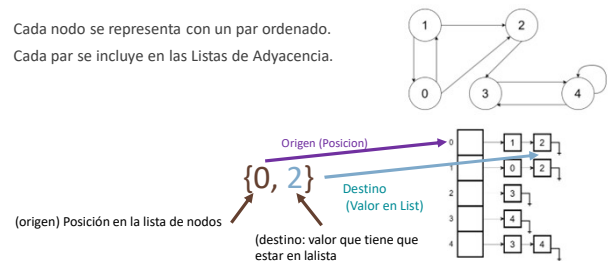


39

39

Listas de Adyacencia (Dirigidas)

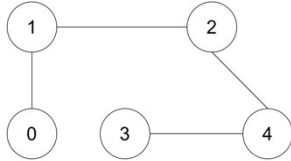
- Cada nodo se representa con un par ordenado.
- Cada par se incluye en las Listas de Adyacencia.



40

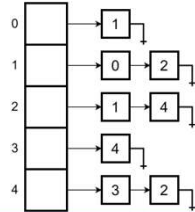
Listas de adyacencia (no dirigidas)

Grafo:



$V = \{0, 1, 2, 3, 4\}$
 $E = \{ \{0, 1\}, \{1, 2\}, \{2, 4\}, \{3, 4\} \}$

Lista de adyacencia:

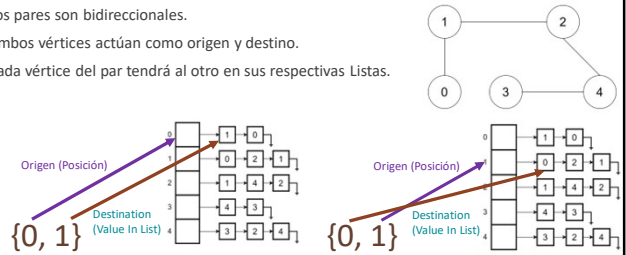


41

41

Listas de adyacencia (no dirigidas)

- Los pares son bidireccionales.
- Ambos vértices actúan como origen y destino.
- Cada vértice del par tendrá al otro en sus respectivas Listas.

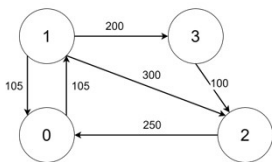


42

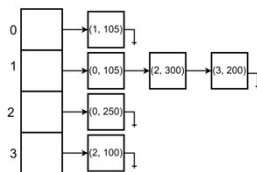
42

Listas de adyacencia (ponderadas)

Grafo:



Lista de adyacencia:



43

43

Práctica

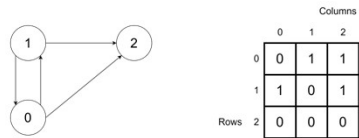
Lista de adyacencia

44

44

Matriz de adyacencia

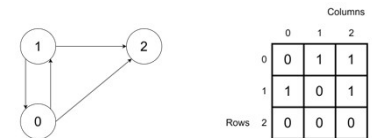
- En lugar de usar listas, trabajamos exclusivamente con una matriz 2D.
- Usamos una matriz $N \times N$, donde N es el número de vértices en el grafo. ($N = |V|$)
- Las filas y columnas representan los vértices.



45

MATRIZ DE ADYACENCIA

- Cada fila y columna representan vértices en el grafo.
- Lo leemos por fila.
- Cada columna distinta de cero indica que el vértice es adyacente.

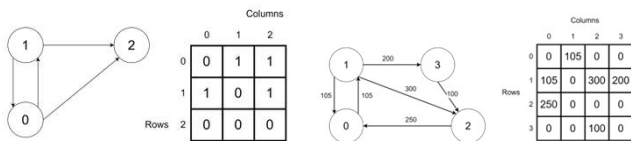


46

Matriz de adyacencia

Ejemplo:

- La fila 1 indica los vértices adyacentes al vértice 1.
- Las columnas 0 y 2 son distintas de cero. Por lo tanto, son adyacentes a 1.
- Cuando el grafo no está ponderado, usamos 0 y 1 (o booleanos).
- De lo contrario, usamos los pesos.

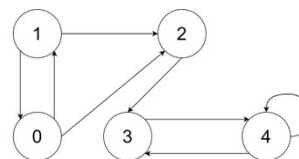


47

Matriz de Adyacencia (Dirigida)

Grafo:

Matriz de adyacencia:



$V = \{0, 1, 2, 3, 4\}$

$E = \{(0, 1), \{1, 0\}, \{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 3\}, \{4, 4\}\}$

48

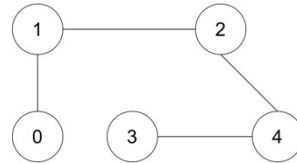
Práctica

Matriz de adyacencia

49

Matriz de adyacencia (no dirigida)

Grafo:



$V = \{0, 1, 2, 3, 4\}$

$E = \{ \{0, 1\}, \{1, 2\}, \{2, 4\}, \{3, 4\} \}$

Matriz de Adyacencia:

	Columns				
	0	1	2	3	4
0	0	1	0	0	0
1	1	0	1	0	0
2	0	1	0	0	1
3	0	0	0	0	1
4	0	0	1	1	0

50

Matriz de adyacencia (no dirigida)

- Cuando un grafo no está dirigido, la matriz es simétrica.
- La información por debajo y por encima de la diagonal es la misma.
- Toda la información que necesitamos está en la diagonal inferior.

Matriz de adyacencia:

	Columns				
	0	1	2	3	4
0	0	1	0	0	0
1	1	0	1	0	0
2	0	1	0	0	1
3	0	0	0	0	1
4	0	0	1	1	0

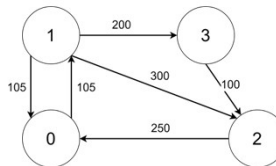
$V = \{0, 1, 2, 3, 4\}$

$E = \{ \{0, 1\}, \{1, 2\}, \{2, 4\}, \{3, 4\} \}$

51

Matriz de adyacencia (ponderada)

Grafo:



Matriz de Adyacencia:

	Columns			
	0	1	2	3
0	0	105	0	0
1	105	0	300	200
2	250	0	0	0
3	0	0	100	0

En este caso usamos 0 para denotar que no hay adyacencia. Hay casos en los que 0 es una ponderación válida, así como valores negativos. ∞ se puede usar en lugar de 0.

52

Comparación de implementaciones

¿Qué implementación es mejor?

- Depende de la densidad del grafo.
- La densidad es la relación entre $|E|$ y $|V|^2$
- Cuando $|E|$ está cerca de $|V|^2$, es un grafo denso.
 - Si $|E|$ es mucho menor que $|V|^2$, es un grafo disperso.

53

53

Comparación de implementaciones

¿Qué implementación es mejor?

- **Listas de adyacencia**
 - Son mejores para grafos dispersos.
 - Al moverse a través de cada vértice y cada borde, su $O(|V| + |E|)$.
 - Su complejidad espacial también es $O(|V| + |E|)$.
- Por qué $|V|$?
 - Recuerde que para simplificar, usamos números enteros para los vértices.
- ¿Qué sucede si los vértices son de otros tipos, como Strings?
 - Primero debemos encontrar el vértice en la matriz.

54

54

Comparación de implementaciones

¿Qué implementación es mejor?

- **Matriz de adyacencia**
 - Son mejores para grafos densos.
 - Al moverse a través de cada vértice y cada arista, su $O(|V|^2)$.
 - Su complejidad espacial también es $O(|V|^2)$.

Por qué $|V|$?

- Recuerde que para simplificar, usamos números enteros para los vértices.
- ¿Qué sucede si los vértices son de otros tipos, como Strings?
 - Primero debemos encontrar el vértice en la matriz.

55

55

Comparación de implementaciones

¿Por qué las listas de adyacencia son mejores para los grafos dispersos?

- Un grafo disperso representado mediante una matriz daría como resultado muchos "espacios vacíos". Ya sean 0 o ∞ entradas.
- Eso podría convertirse en un desperdicio de memoria.
- El uso de la memoria de listas se usa mejor en esa situación, así como algunas operaciones serían más eficientes.
- Cuando un grafo es denso, entonces es más conveniente usar una matriz.
- Como la matriz estaría más llena y el acceso puede ser tan bueno como $O(1)$.

56

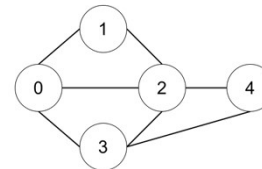
56

Otras implementaciones

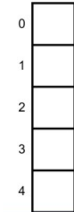
- Otra forma en que podemos implementar un Grafo es usando Mapas.
- La idea sería usar un mapa en lugar de una matriz.
- Cada entrada almacenada en el mapa constaría de: $(V, \text{List}<V>)$.
 - Las keys son el valor del vértice.
 - Los valores son la lista de vértices adyacentes al key.
- Esto es como las listas de adyacencia, pero ahora tenemos acceso $O(1)$ a los vértices. (Tenga en cuenta cuando los vértices no son enteros).

57

Práctica



¿Cuál sería su representación en la Lista de Adyacencia?

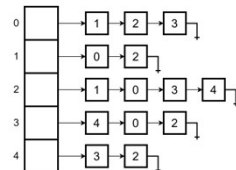
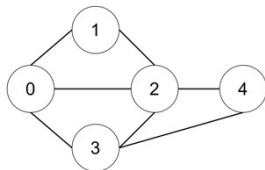


?

58

Práctica

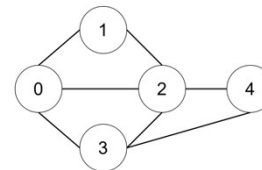
¿Cuál sería su representación en la Lista de Adyacencia?



59

Práctica

¿Cuál sería su representación en la Matriz de Adyacencia?

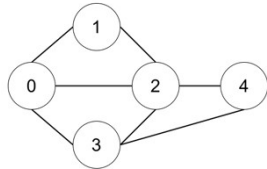


	Columns				
	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

?

60

Práctica



¿Cuál sería su representación en la Matriz de Adyacencia?

	Columns				
	0	1	2	3	4
0	0	1	1	1	0
1	1	0	1	0	0
2	1	1	0	1	1
3	1	0	1	0	1
4	0	0	1	1	0

61

Recorrido de grafos

NO PONDERADOS

62

Recorrido de grafos

- Al igual que los árboles, a menudo tendremos que atravesar todos los vértices del grafo.
- Como antes queremos acceder a cada nodo y, en este caso, queremos evitar entrar en ciclos interminables.
- Para los grafos tenemos 2 algoritmos de búsqueda comunes:
 - Breadth-first search
 - Depth-first search

63

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

Características clave de BFS:

1. Explora nivel por nivel: BFS comienza en un nodo determinado (generalmente llamado nodo de origen) y explora todos sus vecinos inmediatos antes de pasar a sus vecinos, esencialmente explorando el grafo capa por capa.

64

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

Características clave de BFS:

1. Utiliza queues: BFS se basa en una estructura de datos de queues para realizar un seguimiento de los nodos que deben visitarse. Esto garantiza que los nodos se visiten en el orden correcto (primero en entrar, primero en salir). Trabajos en grafos y árboles: BFS se puede aplicar tanto a grafos dirigidos como no dirigidos, así como a estructuras de árbol.

65

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

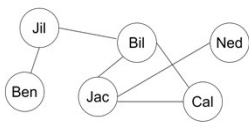
Características clave de BFS:

1. No recursivo: BFS se implementa normalmente de forma iterativa mediante una fila, a diferencia de la búsqueda en profundidad (DFS), que a menudo se implementa de forma recursiva.

66

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

- Comenzamos en un vértice inicial. (Puede ser cualquier nodo)
- Luego visitamos todos los nodos que están adyacentes a él.
- Luego visitamos los nodos adyacentes a esos, y así sucesivamente.
- Esto se repite hasta que se visitan todos los nodos.

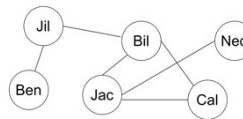


Supongamos que comenzamos en Jil.
El orden de visita sería el siguiente::
{Jil, Ben, Bil, Jac, Cal, Ned}

67

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

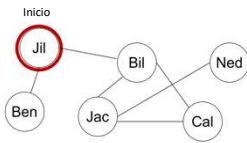
- Este es un algoritmo de ruta más corta.
- Dado que el grafo no está ponderado, todas las aristas tienen la misma "distancia".
- Prioriza todos los nodos más cercanos al inicio, que son sus nodos adyacentes.



Presumiendo que comenzamos con Jil.
El orden de visita sería:
{Jil, Ben, Bil, Jac, Cal, Ned}

68

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

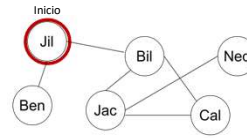


visitado = { }
toExplore = { }

- La forma más sencilla de aplicar estos algoritmos es utilizando colas.
- visitado** – Nodos ya visitados
- toExplore** – Nodos que quedan por visitar.
- La idea es cuando visitamos un nodo:
 - Lo añadimos a visitado.
 - Añadimos todos sus nodos adyacentes a toExplore si no se han visitado y aún no están en el queue.

69

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

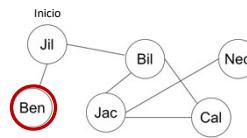


visitado = { Jil }
toExplore = { Ben, Bil }

- Empezamos en Jil.
- Por lo tanto, lo añadimos a visitado.
- Añadimos sus vértices adyacentes a toExplore.
- El orden de inserción depende de una prioridad preestablecida. Por ejemplo, la prioridad podría ser el nodo con el valor más pequeño.
- Para simplificar, preferimos el nodo más a la izquierda.

70

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

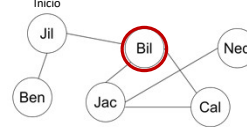


visitado = { Jil, Ben }
toExplore = { Bil }

- Ahora visitamos a Ben.
- El nodo al principio del queue.
- El mismo proceso que antes:
 - Añadir Ben a visitado.
 - Su único nodo adyacente es Jil, que ya fue visitado.
- No se añade nada a toExplore.

71

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

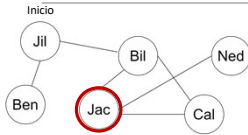


visitado = { Jil, Ben, Bil }
toExplore = { Jac, Cal }

- Ahora visitamos Bil.
- El nodo al principio del queue.
- El mismo proceso que antes:
 - Añadir Bil a los visitados.
 - Jac y Cal se añaden a toExplore.
 - Jac va primero en la fila, ya que es el nodo más a la izquierda.

72

Búsqueda en amplitud (BREADTH-FIRST SEARCH)



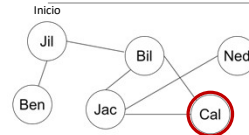
- Ahora visitamos Jac.
- El nodo al principio del queue.
- El mismo proceso que antes:
 - Añadir Jac a los visitados.
 - Ned se agrega a toExplore.

visitado = { Jil, Ben, Bil, **Jac** }
toExplore = { Cal, Ned }

73

73

Búsqueda en amplitud (BREADTH-FIRST SEARCH)



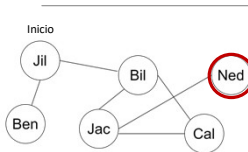
- Ahora visitamos Cal.
- El nodo al principio del queue.
- El mismo proceso que antes:
 - Añadir Cal a los visitados.
 - No se agrega ningún nodo a toExplore.

visitado = { Jil, Ben, Bil, Jac, **Cal** }
toExplore = { Ned }

74

74

Búsqueda en amplitud (BREADTH-FIRST SEARCH)



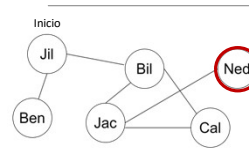
- Ahora visitamos a Ned.
- El nodo al principio del queue.
- El mismo proceso que antes:
 - Añadir Ned a visitado.
 - No se agrega ningún nodo a toExplore.

visitado = { Jil, Ben, Bil, Jac, Cal, **Ned** }
toExplore = { }

75

75

Búsqueda en amplitud (BREADTH-FIRST SEARCH)



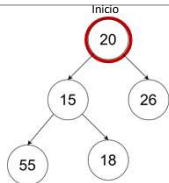
- Ahora que toExplore está vacío, hemos terminado de buscar.
- El orden de visita viene dado por la fila visitado.

visitado = { Jil, Ben, Bil, Jac, Cal, Ned }
toExplore = { }

76

76

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

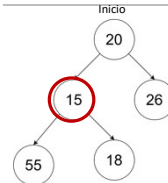


visitado = { 20 }
toExplore = { 15, 26 }

- Si tuviéramos un Árbol, esto es lo mismo que atravesar por nivel.
- Empezamos desde la raíz.
- Añádalo a visitado.
- A continuación, agregue sus elementos secundarios a toExplore.

77

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

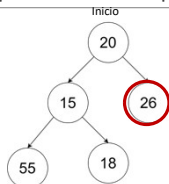


visitado = { 20, 15 }
toExplore = { 26, 55, 18 }

- Continuando con el proceso...

78

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

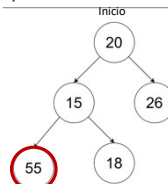


visitado = { 20, 15, 26 }
toExplore = { 55, 18 }

- Continuando con el proceso...

79

Búsqueda en amplitud (BREADTH-FIRST SEARCH)

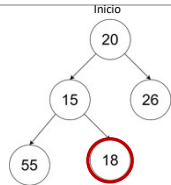


visitado = { 20, 15, 26, 55 }
toExplore = { 18 }

- Continuando con el proceso...

80

Búsqueda en amplitud (BREADTH-FIRST SEARCH)



- Continuando con el proceso...
- El orden de recorrido viene dado por visitados.

Observe que visitamos todos los nodos en un nivel, antes de pasar al siguiente nivel.

visitado = { 20, 15, 26, 55, 18 }
toExplore = { }

81

81

Búsqueda en Profundidad (Depth-First Search)

- **Búsqueda en Profundidad (DFS)** es un algoritmo de recorrido de grafos utilizado para explorar sistemáticamente los nodos y bordes de un grafo.
- Se enfoca en explorar un camino completamente antes de retroceder para explorar otros

82

82

DFS funciona mediante:

- Visita de Nodo
- Profundiza en un vecino
- Continúa hasta que no haya más vecinos
- Retrocede
- Repite

La estructura de datos central detrás de DFS es el stack (explícita o mediante recursión).

83

83

Búsqueda en Profundidad (Depth-First Search)

Características clave del DFS:

1. Explora en profundidad primero:

El DFS se adentra profundamente en una rama del grafo/árbol antes de retroceder para explorar otras ramas.

2. Usa un stack:

El DFS se implementa utilizando un stack (explícita o implícita con recursión). El stack asegura que se exploren primero los vecinos del último nodo visitado.

84

84

Búsqueda en Profundidad (Depth-First Search)

Características clave del DFS:

3. Recursivo o Iterativo:

El DFS se puede implementar de dos maneras:

1. **Recursiva:** Usando un stack de llamadas de funciones.

2. **Iterativa:** Usando una estructura de stack explícita.

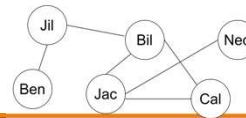
4. Funciona en grafos y árboles:

El DFS se puede aplicar tanto en grafos dirigidos como no dirigidos, así como en estructuras de árboles.

85

Búsqueda en Profundidad (Depth-First Search)

- Comenzamos en un vértice inicial. (Puede ser cualquier nodo)
- Elegimos un nodo adyacente y nos movemos a él.
- Esto se repite hasta llegar a un callejón sin salida.
- Luego vamos a un nodo que no ha sido visitado.



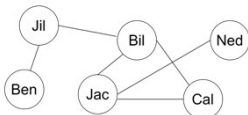
Supongamos que comenzamos en Jil.
El orden de visita sería el siguiente::
{Jil, Bil, Cal, Jac, Ned, Ben}

86

86

Búsqueda en Profundidad (Depth-First Search)

- El objetivo de este algoritmo es seguir una ruta lo más lejos posible sin repetir nodos.

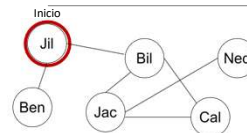


Supongamos que comenzamos en Jil.
El orden de visita sería el siguiente::
{Jil, Bil, Cal, Jac, Ned, Ben}

87

87

Búsqueda en Profundidad (Depth-First Search)



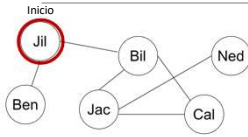
visitado = { }
toExplore = { }

- La forma más sencilla de aplicar estos algoritmos es utilizando un **queue** y un **stack**.
- **visitados** – Nodos ya visitados.
- **toExplore** – Nodos que quedan por visitar.
- La idea es cuando visitamos un nodo:
 - Lo añadimos a visitado.
 - Agregamos todos sus nodos adyacentes a toExplore si no se han visitado y aún no están en la pila.

88

88

Búsqueda en Profundidad (Depth-First Search)

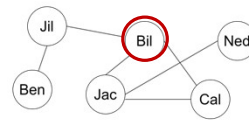


visitado = { Jil }
toExplore = { Bil, Ben }

- Empezamos en Jil.
- Por lo tanto, lo añadimos a visitado.
- Añadimos sus nodos adyacentes a toExplore.
- Al igual que antes, el orden está preestablecido.
- Priorizaremos agregar primero el nodo más a la izquierda.
- **Recuerde que es un stack!!**

89

Búsqueda en Profundidad (Depth-First Search)

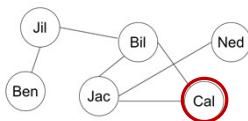


visitado = { Jil, Bil }
toExplore = { Cal, Jac, Ben }

- Visitamos Bil.
- Por lo tanto, lo añadimos a visitado.
- Añadimos sus nodos adyacentes a toExplore.
- Al igual que antes, Jil no se agrega porque ya fue visitado.
- Añadimos Jac y Cal (en ese orden).
- **Stacks!**

90

Búsqueda en Profundidad (Depth-First Search)

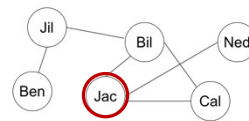


Visitado = { Jil, Bil, Cal }
toExplore = { Jac, Ben }

- Visitamos Cal.
- Por lo tanto, lo añadimos a visitado.
- Añadimos sus nodos adyacentes a toExplore.
- Bil no se añade ya que han sido visitados.
- Jac no se agrega porque ya está en el stack.

91

Búsqueda en Profundidad (Depth-First Search)

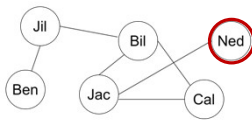


Visitado = { Jil, Bil, Cal, Jac }
toExplore = { Ned, Ben }

- Visitamos Jac.
- Por lo tanto, lo añadimos a visitado.
- Añadimos sus nodos adyacentes a toExplore.

92

Búsqueda en Profundidad (Depth-First Search)

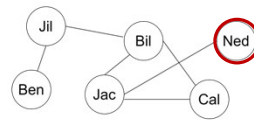


- Visitamos a Ned.
- Por lo tanto, lo añadimos a visitado.
- Añadimos sus nodos adyacentes a toExplore.

Visitado = { Jil, Bil, Cal, Jac, Ned }
toExplore = { Ben }

93

Búsqueda en Profundidad (Depth-First Search)

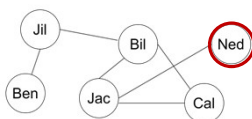


- Visitamos Ben.
- Por lo tanto, lo añadimos a visitado.
- Añadimos sus nodos adyacentes a toExplore.

Visitado = { Jil, Bil, Cal, Jac, Ned, Ben }
toExplore = { }

94

Búsqueda en Profundidad (Depth-First Search)

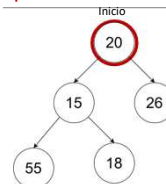


- toExplore está vacío, por lo tanto, hemos terminado de buscar.
- El orden de recorrido viene dado por visitado.

Visitado = { Jil, Bil, Cal, Jac, Ned, Ben }
toExplore = { }

95

Búsqueda en Profundidad (Depth-First Search)



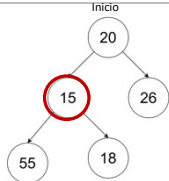
- Si tuviéramos un árbol, esto se trata como un recorrido en pre-orden.
- Empezamos desde la raíz.
- Añádalo a visitado.
- A continuación, agregue sus elementos secundarios a toExplore.
- En este caso, agreguemos primero el hijo derecho al stack.

visitado = { 20 }
toExplore = { 15, 26 }

96

Búsqueda en Profundidad (Depth-First Search)

- Continuando el proceso...

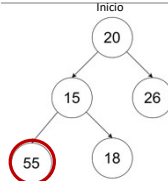


visitado = { 20, 15 }
toExplore = { 55, 18, 26 }

97

Búsqueda en Profundidad (Depth-First Search)

- Continuando el proceso...

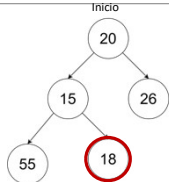


visitado = { 20, 15, 55 }
toExplore = { 18, 26 }

98

Búsqueda en Profundidad (Depth-First Search)

- Continuando el proceso...

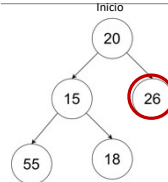


visitado = { 20, 15, 55, 18 }
toExplore = { 26 }

99

Búsqueda en Profundidad (Depth-First Search)

- Continuando el proceso...
- El orden de recorrido viene dado por visitado.



Visitado = { 20, 15, 55, 18, 26 }
toExplore = { }

Observe que el orden es el mismo que si hubiéramos recorrido en pre-order.

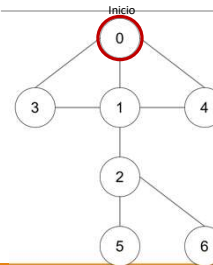
100

Búsqueda en Profundidad (Depth-First Search)

- Por lo general, es preferible agregar primero el extremo derecho al stack, tal como lo hicimos con el árbol.
- De esa manera, siempre visitamos primero el nodo más a la izquierda.

101

Práctica

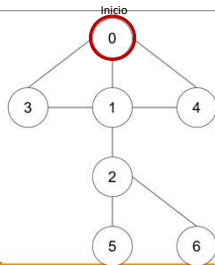


¿Cuál sería el recorrido de la Amplitud Primero??

Supongamos que primero atravesamos hasta el vecino más a la izquierda.

102

Práctica



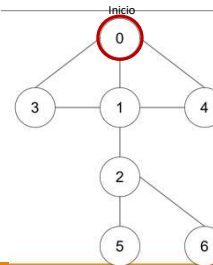
¿Cuál sería el recorrido de la Amplitud Primero??

Supongamos que primero atravesamos hasta el vecino más a la izquierda.

{ 0, 3, 1, 4, 2, 5, 6 }

103

Práctica

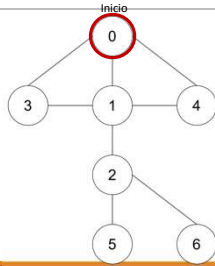


¿Cuál sería la travesía de Profundidad-Primero??

Supongamos que primero recorremos agregando el vecino más a la derecha.

104

Práctica



¿Cuál sería la travesía de Profundidad-Primero?
Supongamos que primero recorremos agregando el vecino más a la derecha.
{ 0, 3, 1, 2, 5, 6, 4 }

105

Recorrido de grafos

- Estos algoritmos se utilizan principalmente para grafos no ponderados.
- Ambos recorridos funcionan tanto si el grafo es dirigido como si no está dirigido.
- Cuando se pondera un grafo, normalmente utilizamos diferentes algoritmos que tienen en cuenta el peso.

106

Recorrido de grafos

PONDERADOS

107

Algoritmos para gráficos ponderados

- A menudo, cuando se pondera un grafo, es porque esa información es relevante para los datos que se representan.
- Una necesidad común de estos grafos es encontrar la distancia más corta (pesos) entre los nodos.
- Esto a menudo es necesario para resolver problemas como encontrar la distancia más corta del punto A al punto B.
- Un ejemplo serían las aerolíneas que intentan ahorrar combustible para encontrar el camino más corto de una ciudad a otra.

108

El camino más corto

- Cuando tenemos un grafo no ponderado, nos preocupamos principalmente por visitar todos los vértices.
- O encontrar un camino hacia un vértice específico desde el punto de partida.
- Algoritmos como Breath-First Search intentan encontrar el camino más corto hacia todos los nodos (o uno específico) desde el punto de partida permaneciendo lo más cerca posible del punto de partida.

109

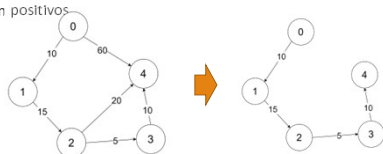
El camino más corto

- En los grafos ponderados queremos encontrar el camino más corto desde el nodo inicial hasta TODOS los demás nodos en la "distancia" más corta posible.
- La distancia está determinada por el peso en cada arista.
- El algoritmo más común es el algoritmo de **Dijkstra**.

110

1. Algoritmo de Dijkstra

- **Objetivo:** Encontrar el camino más corto a cada vértice desde el punto de partida.
- Se supone que los pesos son positivos.
- Para ello necesitamos:
 - 2 sets: S y S-V
 - 2 arrays: d y p



111

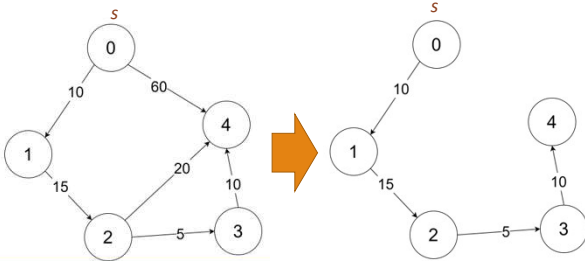
Algoritmo de Dijkstra

- **Asumir:**
 - **v** - un vértice del grafo
 - **s** - Vértice del que partimos.
- **Sets:**
 - **S** - Tiene todos los nodos que se han calculado
 - **S-V** - Vértices que necesitan computación
- **Arrays:**
 - **d[v]** - Tendrá la distancia más corta actual para el vértice v.
 - **p[v]** - Tendrá predecesor de v en el camino hacia v desde s.

112

Algoritmo de Dijkstra

Probemos estos algoritmos para este grafo:



113

Algoritmo de Dijkstra

Visualize: Comenzamos en el vértice s . Por ahora, no hay otros nodos visibles en el grafo.

Inicialmente:

- S solo tiene el nodo de inicio s .

- $V - S$ tiene todos los demás nodos.

Las matrices se llenarán a continuación.

$$S = \{0\}$$

$$V - S = \{1, 2, 3, 4\}$$

v	$d[v]$	$p[v]$
1	-	-
2	-	-
3	-	-
4	-	-



114

Algoritmo de Dijkstra

Visualize: Ahora expandimos el grafo, en función de quién es adyacente a s .

Llenamos las matrices:

- $d[v]$ - tiene distancia de los nodos adyacentes y ∞ para los demás.

- $p[v]$ - comienza en s .

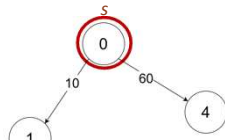
$$S = \{0\}$$

$$V - S = \{1, 2, 3, 4\}$$

v	$d[v]$	$p[v]$
1	10	0
2	∞	0
3	∞	0
4	60	0

Distancia del vértice 1 a s
Su antecesor es 0 (s)

El vértice 2 no es adyacente a s .
Por lo tanto, su distancia es ∞ .



115

Algoritmo de Dijkstra

Proceso:

- Elegimos el vértice u con la distancia más corta a s .

- Para todos los vértices v , si v y u son adyacentes:

- Si $d[v] > w(u, v) + d[u]$, entonces $d[v] = w(u, v) + d[u]$.

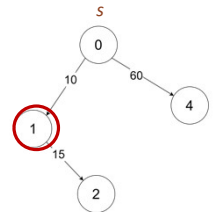
- $w(u, v)$ = peso para el borde $\{u, v\}$.

- Añadimos u a S , y lo eliminamos de $V - S$.

$$S = \{0\}$$

$$V - S = \{1, 2, 3, 4\}$$

v	$d[v]$	$p[v]$
1	10	0
2	∞	0
3	∞	0
4	60	0



116

Algoritmo de Dijkstra

Visualize: Elegimos 1 como u ya que es el más cercano. Nos desplazamos a él y ampliamos el grafo para incluir sus nodos adyacentes.

Su único nodo adyacente es el 2.

Es $d[2] > w(1, 2) + d[1]$?

$= \infty > 15 + 10$? **TRUE**

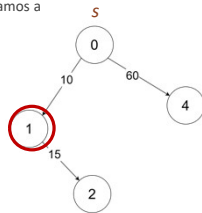
Entonces $d[2] = 25$, y $p[2] = 1$

Añadimos 1 a S .

$S = \{0, 1\}$

$V - S = \{2, 3, 4\}$

v	$d[v]$	$p[v]$
1	10	0
2	25	1
3	∞	0
4	60	0



117

Algoritmo de Dijkstra

Próximo u es 2, debido a que tiene la distancia más corta de los nodos que quedan.

Si $d[3] > w(2, 3) + d[2]$? $\infty > 30$? **TRUE**

Si $d[4] > w(2, 4) + d[2]$? $60 > 45$? **TRUE**

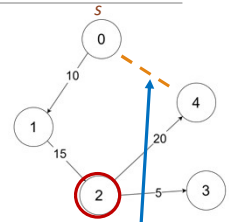
Entonces $d[3] = 30$, y $d[4] = 45$. $p[3] = p[4] = 2$

Sumamos 2 a S .

$S = \{0, 1, 2\}$

$V - S = \{3, 4\}$

v	$d[v]$	$p[v]$
1	10	0
2	25	1
3	30	2
4	45	2



Visualize: Descartamos $\{0, 4\}$ ya que no es parte de la ruta más corta.

118

Algoritmo de Dijkstra

Próximo u es 3.

Es $d[4] > w(3, 4) + d[3]$? $45 > 40$? **TRUE**

Entonces $d[4] = 40$.

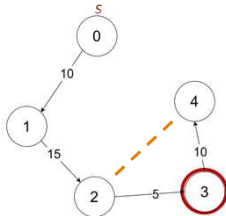
Añadimos 3 a S . $p[4] = 3$.

Visualize: Descartamos $\{2, 4\}$, porque ya no es parte de la ruta más corta.

$S = \{0, 1, 2, 3\}$

$V - S = \{4\}$

v	$d[v]$	$p[v]$
1	10	0
2	25	1
3	30	2
4	40	3



119

Algoritmo de Dijkstra

Próximo u es 4.

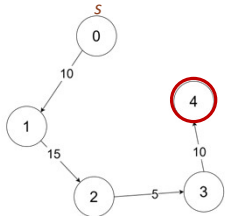
El vértice 4 no tiene nodos adyacentes.

Añadimos 4 a S .

$S = \{0, 1, 2, 3, 4\}$

$V - S = \{\}$

v	$d[v]$	$p[v]$
1	10	0
2	25	1
3	30	2
4	40	3



120

Algoritmo de Dijkstra

Próximo u es 4.

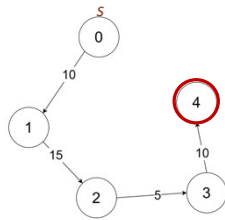
El vértice 4 no tiene nodos adyacentes.

Añadimos 4 a S .

$V - S$ está vacío, terminamos.

$S = \{0, 1, 2, 3, 4\}$
 $V - S = \{\}$

v	$d[v]$	$p[v]$
1	10	0
2	25	1
3	30	2
4	40	3



121

Algoritmo de Dijkstra - Pseudocódigo

DIJKSTRA(Grafo, fuente)

u = vértice sin visitar
 con la distancia más pequeña

para cada vértice v en el grafo
 $distancia[v] = \text{infinito}$
 $visitado[v] = \text{falso}$

$visitado[u] = \text{cierto}$

$distancia[\text{fuente}] = 0$

para cada vecino v de u

repetir hasta que todos los vertices sean visitados

if $visitado[v]$ es falso AND
 $distancia[u] + peso(u,v) < distancia[v]$

u = vértice sin visitar

$distancia[v] = distancia[u] + peso(u,v)$

return distancia

122

Dijkstra Pasos del algoritmo:

1. Inicializar:

- Establezca la distancia al nodo de origen como 0 y todos los demás como ∞ .
- Marcar **todos** los nodos como no visitados.
- Utilice **priority queue** (a menudo un **min heap**) para realizar un seguimiento de los nodos con la distancia más pequeña conocida.

123

Dijkstra Pasos del algoritmo:

2. Visite el nodo actual (Empezando por el origen):

- Por cada vecino **no** visitado:
 - Calcular la distancia tentativa desde el origen.
 - Si esta distancia tentativa es **menor** que la registrada anteriormente, actualícela.

124

123

124

Dijkstra Pasos del algoritmo:

3. Marcar el nodo actual como visitado:

- Un nodo visitado no se volverá a comprobar.

4. **Seleccione el siguiente nodo** con la distancia tentativa más pequeña y repita el proceso.

5. **Repita** hasta que se visiten todos los nodos o se encuentre la ruta más corta al nodo deseado.

125

125

2. Árbol de expansión mínimo

- Un árbol de expansión (Spanning Tree) es un subconjunto de aristas, donde todos los vértices están conectados y se pueden acceder a ellos con el número mínimo de aristas.
- Un árbol de expansión mínimo (Minimum Spanning Tree) es un árbol de expansión (Spanning Tree) donde se minimiza la suma de todos los pesos.
- Esto nos ayuda a encontrar redes que requieran el menor número de conexiones.
- Ejemplo: si estuviéramos diseñando el sistema eléctrico entre ciudades, esto podría ayudar a minimizar el costo.

126

126

Algoritmos

- Algoritmo de Prim (El algoritmo de Jarník en Europa)
- Algoritmo de Kruskal
- Algoritmo de Borůvka

127

127

Algoritmo de Prim

- **Objetivo:** Encuentre el árbol de expansión mínimo (Minimum Spanning Tree)
- Los pesos pueden ser tanto positivos como negativos. Utilizaremos pesos positivos.
- El grafo debe ser no dirigido.
- Para ello necesitamos:
 - 2 sets: S y S-V
 - 2 arrays: d y p

128

128

Un árbol de expansión es un subconjunto del grafo que:

- Conecta todos los vértices
- No tiene ciclos
- Utiliza exactamente $V - 1$ aristas (donde V = número de vértices)
- Un árbol de expansión mínimo es el árbol generado con el peso total de aristas más pequeño posible.

129

129

Algoritmo de Prim

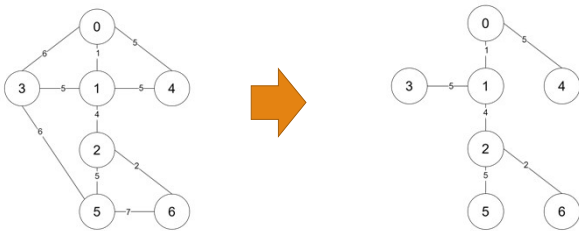
- Presuma:
 - v - un vértice del grafo
 - s - Vértice del que partimos.
- Sets:
 - S - Tiene todos los nodos que se han calculado
 - $V - S$ - Vértices que necesitan ser calculados
- Arrays:
 - $d[v]$ - Tendrá la longitud de la arista final.
 - $p[v]$ - Tendrá predecesor de v en el camino hacia v desde s .

130

130

Algoritmo de Prim

Probemos estos algoritmos para este grafo:



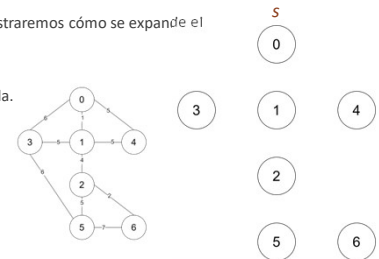
131

131

Algoritmo de Prim

- Para facilitar el proceso, solo mostraremos cómo se expande el grafo.
- Y cómo cambian S y $V - S$.
- Elegimos 0 como punto de partida.

$S = \{0\}$
 $V - S = \{1, 2, 3, 4, 5, 6\}$



132

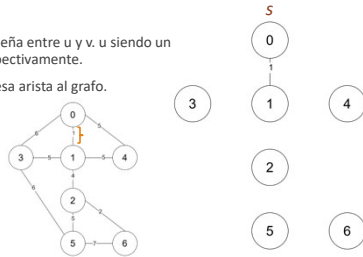
132

Algoritmo de Prim

Proceso:

- Buscamos la arista más pequeña entre u y v , u siendo un vértice en S y v en $V - S$, respectivamente.
- Es la arista $\{0, 1\}$, añadimos esa arista al grafo.
- Añadimos 1 a S .

$S = \{0, 1\}$
 $V - S = \{2, 3, 4, 5, 6\}$



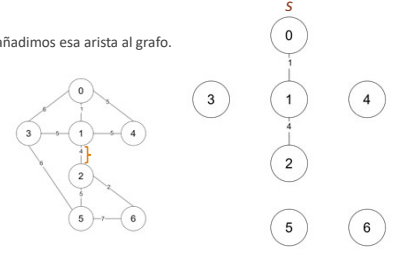
133

Algoritmo de Prim

Proceso:

- Arista $\{1, 2\}$ es la menor, añadimos esa arista al grafo.
- Añadimos 2 a S .

$S = \{0, 1, 2\}$
 $V - S = \{3, 4, 5, 6\}$



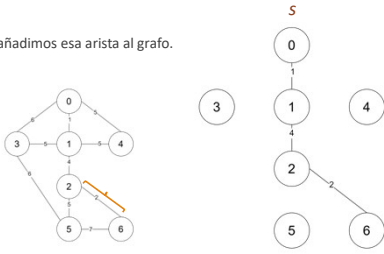
134

Algoritmo de Prim

Proceso:

- Arista $\{2, 6\}$ es la menor, añadimos esa arista al grafo.
- Añadimos 6 a S .

$S = \{0, 1, 2, 6\}$
 $V - S = \{3, 4, 5\}$



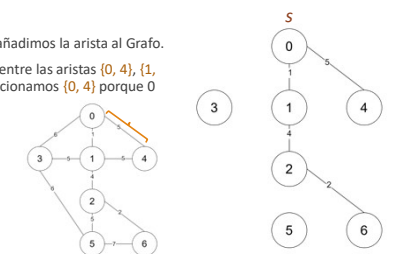
135

Algoritmo de Prim

Proceso:

- Arista $\{0, 4\}$ es la menor, añadimos la arista al Grafo.
- Note que hay un empate entre las aristas $\{0, 4\}$, $\{1, 3\}$, $\{1, 4\}$, and $\{2, 5\}$. Seleccionamos $\{0, 4\}$ porque 0 estaba en S primero.
- Añadimos 4 a S .

$S = \{0, 1, 2, 6, 4\}$
 $V - S = \{3, 5\}$



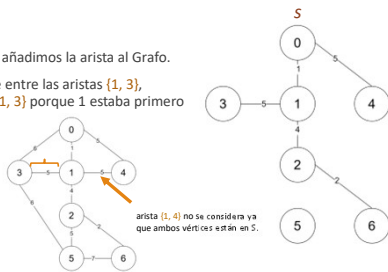
136

Algoritmo de Prim

Proceso:

- Arista $\{1, 3\}$ es la menor, añadimos la arista al Grafo.
- Note que hay un empate entre las aristas $\{1, 3\}$, y $\{2, 5\}$. Seleccionamos $\{1, 3\}$ porque 1 estaba primero en S.
- Añadimos 3 a S.

$S = \{0, 1, 2, 6, 4, 3, 5\}$
 $V - S = \{5\}$



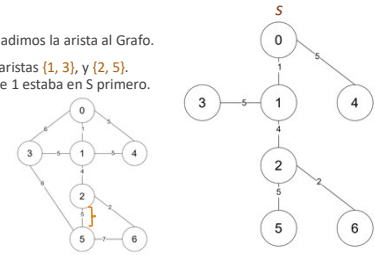
137

Algoritmo de Prim

Proceso:

- Arista $\{1, 3\}$ es la menor, añadimos la arista al Grafo.
- Había un empate entre las aristas $\{1, 3\}$, y $\{2, 5\}$. Seleccionamos $\{1, 3\}$ porque 1 estaba en S primero.
- Añadimos 3 a S.

$S = \{0, 1, 2, 6, 4, 3, 5\}$
 $V - S = \{5\}$



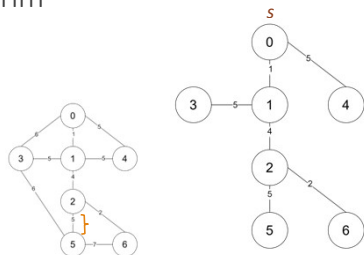
138

Algoritmo de Prim

$V - S$ está vacío

Por lo tanto, terminamos.

$S = \{0, 1, 2, 6, 4, 3, 5\}$
 $V - S = \{5\}$



139

Algoritmo de Prim Pseudocódigo

- Elegir un vértice inicial
- Marca que fue visitado
- Añadir todas sus aristas a minimum priority queue
- Mientras el MST no está completo eliminar la arista más pequeña
 si el vértice de destino no está visitado:
 añadir arista a MST
 marcar Vértice visitado
 añadir nuevas aristas de vértice al queue

140

Diferencias entre Prim y Dijkstra

Algoritmo de Dijkstra

- Encuentra la ruta más corta (**shortest path**) desde un nodo inicial (tiene que ser dado).
- Trabaja en grafos dirigidos y no dirigidos.
- No trabaja con pesos negativos.

Algoritmo de Prim

- Encuentra el árbol de expansión mínimo (**Minimum Spanning Tree**) (puede comenzar en cualquier nodo). Construye un subconjunto de aristas que conecta todos los nodos con el **peso total mínimo de aristas** y sin ciclos
- Trabaja con grafos no dirigidos.
- Los pesos pueden ser negativos.

141

141

Algoritmo de Prim

Característica	Matriz de Adyacencia	Lista de adyacencia
Almacenamiento	Matriz 2D	Listas
Memoria	$O(V^2)$	$O(V + E)$
Mejor para	Grafos densos	Grafos dispersos
Complejidad Prim	$O(V^2)$	$O(E \log V)$

142

Algoritmo de Dijkstra

Característica	Matriz de Adyacencia	Lista de adyacencia
Almacenamiento	Matriz 2D	Listas
Memoria	$O(V^2)$	$O(V + E)$
Mejor para	Grafos densos	Grafos dispersos
Complejidad de Dijkstra	$O(V^2)$	$O(E \log V)$

143

Conclusiones

ALMOST DONE

144

144

Conclusiones

Grafos es una estructura de datos muy genérica.

Existen 2 formas comunes de implementar grafos.

- Listas de adyacencia y matriz de adyacencia

Algoritmos para buscar en grafos sin ponderación.

En un grafo ponderado:

- Podemos encontrar el camino más corto. (Dijkstra)

- Podemos conseguir el Árbol Mínimo Expansivo. (Prism)

145

145

146

146